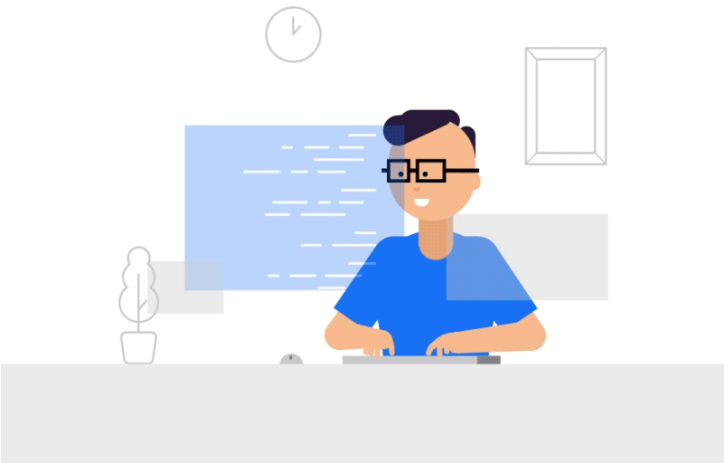
01 Python basics

Scripting

Elke Boonen, Dries Geysens & Tristan Vandevelde

Learning objectives (ECTS)



- Learn the **syntax of Python**
- **Explore possibilities** regarding working with **data** (file, web API, web scraping...)
- Create **scripts**
- **Automate** processes

Learning materials

- **Canvas** LMS https://thomasmore.instructure.com/
  - Presentations & Assignments (CodeGrade)
  - E-Book: Automate the boring stuff with Python (https://automatetheboringstuff.com)
- **Online** https://www.python.org/ and https://pypi.org/ (search packages)
- **Github** Scripting (code from slides & extra notebooks!)

  & Scripting-Students (code from class)
- **Software**
  - https://www.python.org/downloads/ (IDLE Python)
  - https://www.anaconda.com/
  - Love to work with Visual Studio Code? Install Python extensions!

Schedule

Before Easter break	**After Easter break**
01 Python basics	08 Scheduling
02 Python basics	09 Images
03 Python basics	10 Databases
04 Strings & regex	11 Packaging
05 Files (IO)	12 Conclusion/exam prep
06 API	*13 Extra recap*
07 Webscraping	

Schedule is always subject to unexpected circumstances

Evaluation

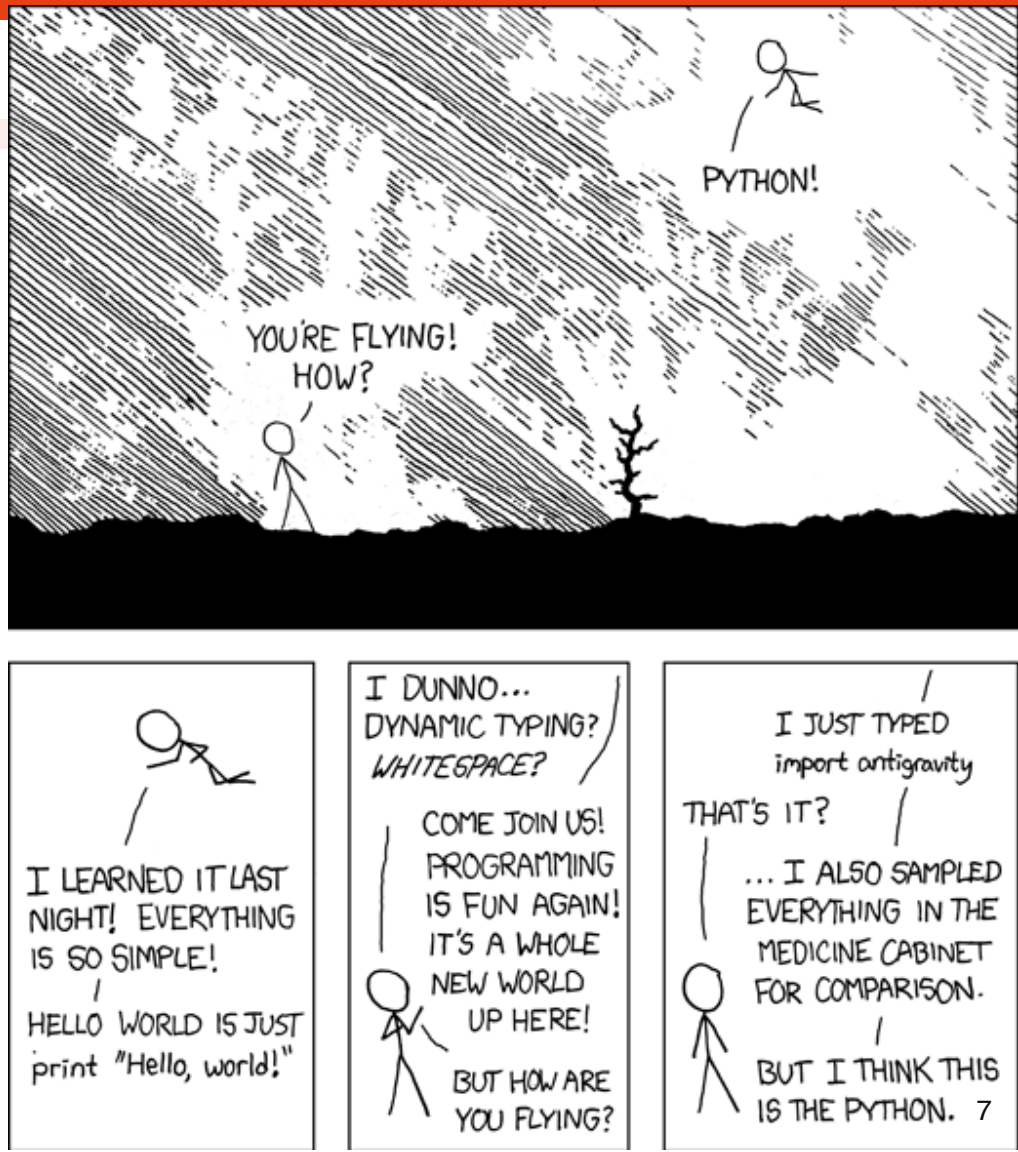
- **1st term**
  - Permanent Evaluation (30 %):
    - CodeGrade exercises (each week)
  - Take-home exam (70 %)
- **2nd term**
  - Computer Exam (100 %)

01 Python basics

- Why Python?
- The basics!
- Flow control
- Functions
- Lists
- Arrays
- Pythonic code

Why Python?

- Python is everywhere
- Python + data = ❤
- Python is easy to learn, clear to read, and simple to code!
- No more ; {
- Monty Python
- Basics of C
- Very powerful but slow



Let the games begin!

- **Download** Python
  - https://www.python.org/ (**latest version 3.9.1**)
    - add Python to PATH

      ☑ Install launcher for all users (recommended)
      ☑ Add Python 3.9 to PATH
  - or Microsoft store
  - or ...
- **Check** version!
  - in shell
  - or start menu
  - or ...

```
PS C:\> python --version
Python 3.9.1
```

Run Python run

- Different ways of running Python commands/scripts
  - **Python IDLE shell**
    - Quickly check 'something'
    - Interactive Python shell, PowerShell...

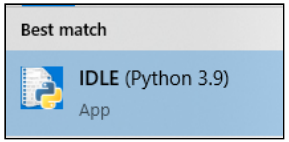
  Python scripts
    - Assignments (upload to CodeGrade)
    - Visual Studio Code, Anaconda...

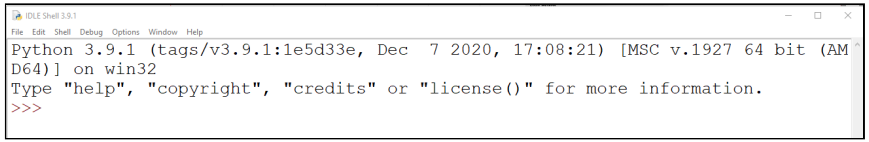
  Python notebooks
    - Script/code in class
    - Visual Studio Code, Anaconda...

Python shell

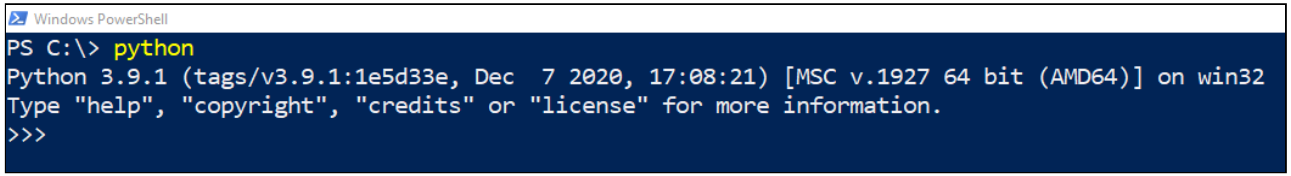
- Open **Python IDLE shell**

  interactive development learning environment





- Or open another shell (**Powershell**) and type **keyword python**



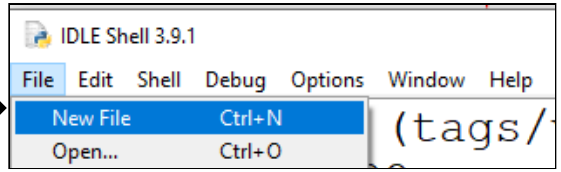
- **Hello world!** Use print-command
  - Experiment with other commands or calculations!

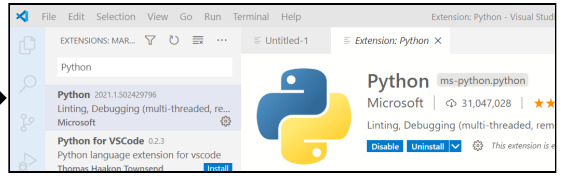
```
Python 3.9.1 (tags/v3.9.1:1e5d33e, Dec  7 2020, 17:08:21) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hello world!")
Hello world!
>>>
```

Python script

- Open IDLE **script editor** (File, new file)
- Or use another editor like **Visual Studio Code** and **install the extension**
- **Save** scripts as **.py-file**
- **Run script**
  - **Hit F5**
  - Use **python3** or python-command

    depends on version you installed!

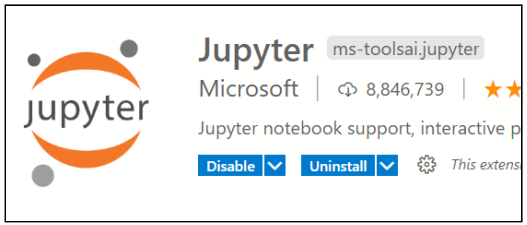


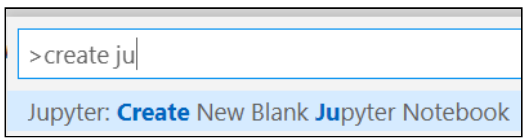


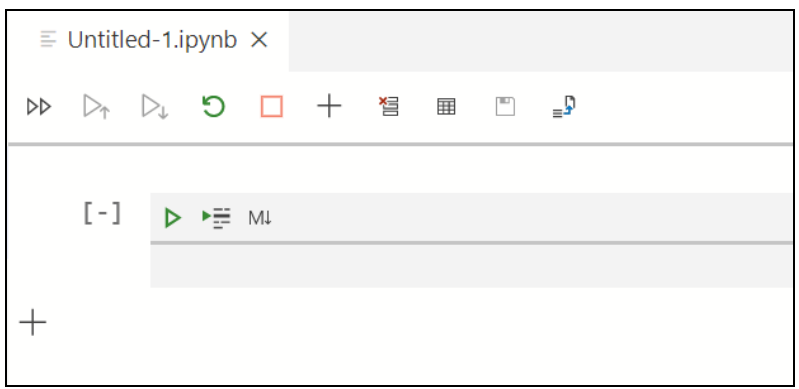
```
Python 3.9.1 (tags/v3.9.1:1e5d33e, Dec  7 2020, 17:08:21) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> python .\helloworld.py
Hello world!
>>>
```

Jupyter notebooks

- Notebook: interactive document with code, comments, visuals...
- Use **Anaconda** (download & install)
- Or use **Visual Studio Code** and **install the extension**
  - Open command palette (Ctrl + Shift + P)
  - Select "Jupyter: Create New Blank Jupyter Notebook
  - save as **.ipynb-file**

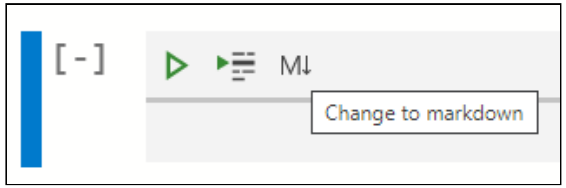


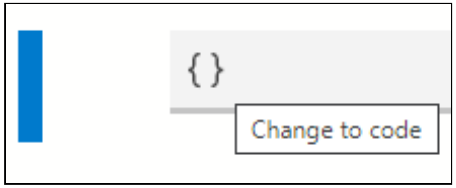


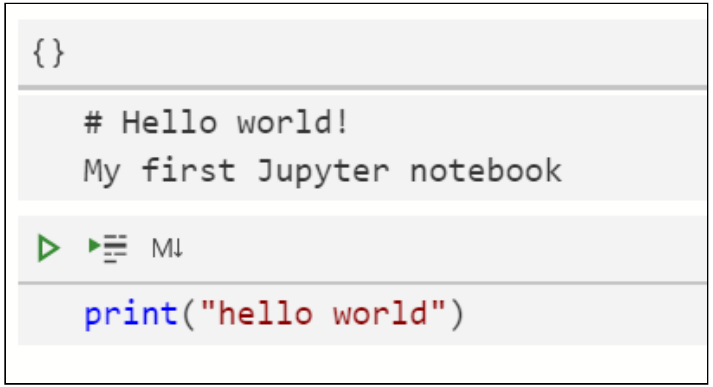


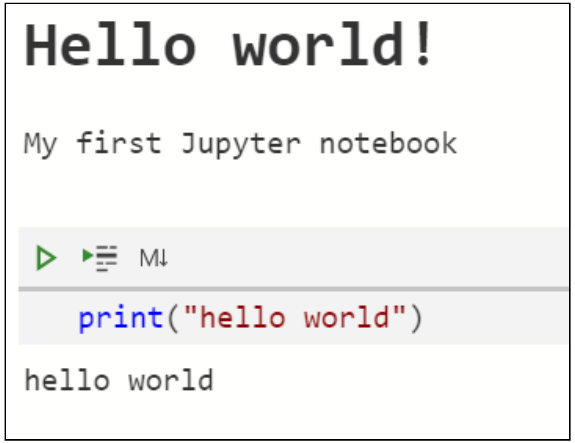
Jupyter notebooks

- The notebook consists of cells: code or markdown (comments)
  - Click cell to change purpose
- Markdown tags
  - take a look at https://towardsdatascience.com/jupyter-and-markdown









Run entire notebook or individual cell ▷

01 Python basics

- ~~Why Python?~~
- The basics!
- Flow control
- Functions
- Lists
- Arrays
- Pythonic code

Python basics: code blocks

- Use # for comments in code
- Every statement on a new line (no ; at the end)
- Use : to start a **block of code**
  - Function definition
  - Control structures
- Statements in block must be indented

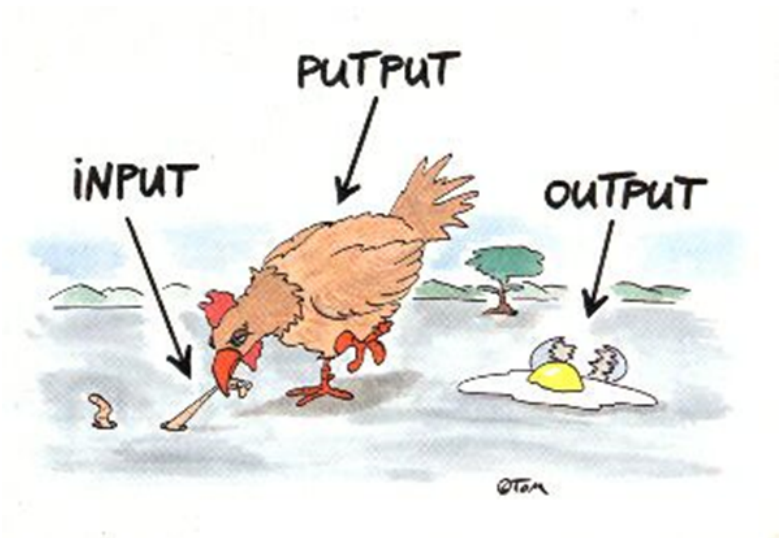
```python
# find sign of x
x = input("Enter x: ")
print("value of x:", x)
print(type(x))

# convert string x to int x
x = int(x)

# check sign
if x > 0:
    print("x is positive")
elif x < 0:
    print("x is negative")
else:
    print("x is zero")
```

Python basics: input & output

- Use **input** to get input
- Use **print** for output
  - use + or , to combine output

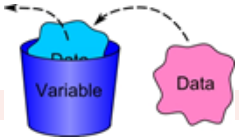



```python
# find sign of x
x = input("Enter x: ")
print("value of x:", x)
print(type(x))

# convert string x to int x
x = int(x)

# check sign
if x > 0:
    print("x is positive")
elif x < 0:
    print("x is negative")
else:
    print("x is zero")
```

Python basics: variables



- Variables = storage in memory
- Create variable with a name (case sensitive!)
  - No need for a type (dynamically-typed)
- Check for type by **type**
- Everything from input = string
- Conversion between types
  - **float**: to decimal
  - **int**: to integer
  - **str**: to string

```python
# find sign of x
x = input("Enter x: ")
print("value of x:", x)
print(type(x))

# convert string x to int x
x = int(x)

# check sign
if x > 0:
    print("x is positive")
elif x < 0:
    print("x is negative")
else:
    print("x is zero")
```

```
Enter x: -1
value of x: -1
<class 'str'>
x is negative
```

Python basics: expressions

- **Variable = expression**
- An **expression:** a value or a combination of values, variables, and operators
- **Possible operators**
  - **Arithmetic** operators +, -, *, /
    - % = modulus (remainder after integer division)
    - ** = exponentiation (x to the power of y)
    - // = integer division
  - **Assignment** operators +=,-=,*=,/=,%=,**=,//=
    - *eg: x += 5 is short version for x = x + 5*
  - **Comparison** operators <,>,>=,<=, !=,==
    - result in a Boolean = True or False (capitalized!)

```python
# assignment
x = 5

# multiple assignments
a, b, c = 1,"ok",3

# arithmetic operators
x = 10
y = 3
print(x+y)
print(x-y)
print(x*y)
print(x/y)
print(x%y)
print(x**y)
print(x//y)

# assignment operators
x *= 5
print(x)
x -= 50
print(x)

# comparison operators
print(x==y)
print(x!=y)
print(x>y)
print(x<y)
print(x>=y)
print(x<=y)
```

```
13
7
30
3.33333333333333
35
1
1000
3
50
0
False
True
False
True
False
True
```

Python basics: built-in functions

- **A lot of built-in functions**
  - string
  - date/time
  - math
  - …

```python
s = "hello"
print(s)
print(s.capitalize())
print(s.upper())
print(s.rjust(7))
print(s.center(7))
print(s.replace('l','(ell)'))
print('  world '.strip())
```

```
hello
Hello
HELLO
  hello
 hello
he(ell)
(ell)o
world
```

- •https://www.w3schools.com/python/python_ref_functions.asp

01 Python basics

- ~~Why Python?~~
- ~~The basics!~~
- Flow control
- Functions
- Lists
- Arrays
- Pythonic code

Python basics: if-structure

- **Conditional statement**
- Execution depends on a condition formed by conditional operators
- if, elif (for every new condition in same if-structure) and else
- Don't forget **:** to start a new block of code!
  - indent all statements depending on the condition

```python
# find sign of x
x = input("Enter x: ")
print("value of x:", x)
print(type(x))

# convert string x to int x
x = int(x)

# check sign
if x > 0:
    print("x is positive")
elif x < 0:
    print("x is negative")
else:
    print("x is zero")
```

Python basics: loops

- **FOR-loop**: iterating over a sequence or a range

  we know exactly how often it will run

- **WHILE-loop**: conditional loop

  loop

```python
# iterating range
for x in range(0,10):
    print("5 *",x,"=",x*5)

# iterating sequence
for x in "something":
    print(x)

# loop as long as x < 5000
# loop count depends on startvalue of x
x = 5
while x < 5000:
    print(x)
    x *= 5
```

```
5 * 0 = 0
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45

s
o
m
e
t
h
i
n
g

5
25
125
625
3125
```

Python basics: combine conditions

- **and**: both conditions are met
- **or**: at least one condition is met
- **not**: reverse result of condition

```python
x=10
y=3

print(x!=y and x<y)
print(x!=y or x<y)
print(not(x<y))
```

```
False
True
True
```

```python
# Sequence of Hailstone
# If n is 1 then the sequence ends.
# If n is even then the next n of the sequence = n/2
# If n is odd then the next n of the sequence = (3 * n) + 1

n = int(input("Enter a number"))
print("n is ",n)

while not (n == 1):     # same as n != 1
    if (n % 2 == 0):
        n/=2
    else:
        n = (3 * n) + 1
    print(n)
```

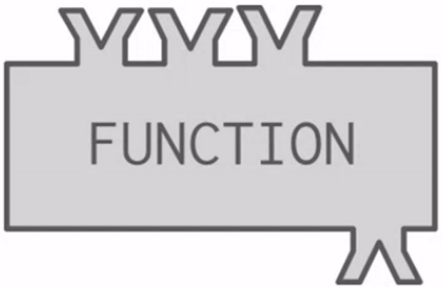
```
n is  5
16
8.0
4.0
2.0
1.0
```

01 Python basics

- ~~Why Python?~~
- ~~The basics!~~
- ~~Flow control~~
- Functions
- Lists
- Arrays
- Pythonic code

Python basics: functions

- A **separate piece of code** only executed when **called**
- Create by keyword **def**
- A function
  - can get values (**arguments**) as input
  - can **return** something back



Python basics: functions

- Don't forget : after function definition to start a code block
- Indent all statements inside function

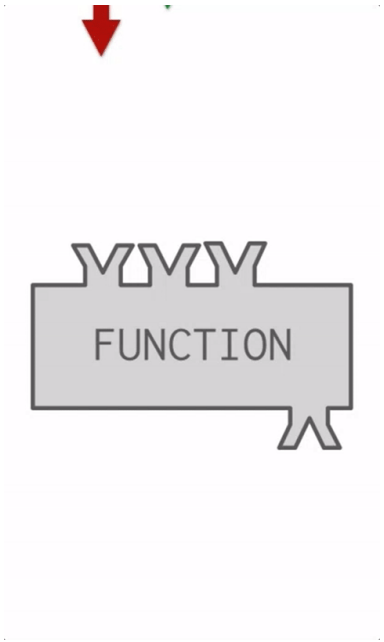
```python
def sign(x):
    if x > 0:
        return "positive"
    elif x < 0:
        return "negative"
    else:
        return "zero"

x = int(input("Enter a number"))
print(x," is ",sign(x))

for x in range(-5,5):
    print(x," is ",sign(x))
```

```
5  is  positive

-5  is  negative
-4  is  negative
-3  is  negative
-2  is  negative
-1  is  negative
0  is  zero
1  is  positive
2  is  positive
3  is  positive
4  is  positive
```



01 Python basics

- ~~Why Python?~~
- ~~The basics!~~
- ~~Flow control~~
- ~~Functions~~
- Collections: lists, tuple, set & dictionary
- Arrays
- Pythonic code

Python basics: collections

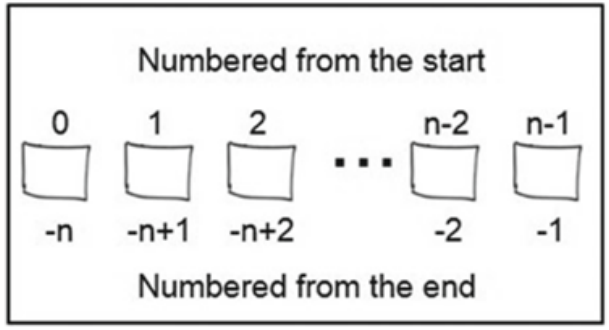
- **Four** different **collection data types**:
  - List = ordered & changeable collection
  - Tuple = ordered & unchangeable collection
  - Set = unordered & changeable collection (does not allow duplicates)
  - Dictionary = ordered & changeable collection (does not allow duplicates)
- **Python uses lists by default**! #hooray
- **Arrays** are a **separate package** so they are still very much in the picture ;) #bummer

Python basics: lists

- A list is a collection of items
- A list is changeable and ordered!
- Create by the usage of **[ ]**
  - A list can hold different types of elements
  - **append** or + to add elements
  - **pop** to remove an element
  - Get length by **len**
- Get elements by index
  - negative indexes!

```python
list = [3, 1, 2]
print(list,list[2])
list[2] = "foo"
list.append("bar")
print(list)
x = list.pop()
print(x,list)
list = list + [8.3, False]
print(list)
print(list[-1])
```

```
[3, 1, 2] 2
[3, 1, 'foo', 'bar']
bar [3, 1, 'foo']
[3, 1, 'foo', 8.3, False]
False
```



Python basics: lists

- Loop a list with a **for-loop**
- Create a list
  - with a range
  - with a for-loop

```
for x in list:
    print(x)

nrs = range(0,5)
squares = [x ** 2 for x in nrs]
print (squares)
```

```
3
1
foo
8.3
False

[0, 1, 4, 9, 16]
```

- Check if an element exists in the list, use **in**

```
print("foo" in list)
print("foo" not in list)
print("bar" in list)

if False in list:
    print(True);
```

```
True
False
False

True
```

- **A string is also a list (of characters)**

Python basics: 2D lists

- **2D lists**? Oh yes!

```
1 list_of_lists = [ [1,2,3], [4,5,6], [7,8,9]]
2 for list in list_of_lists:
3     for x in list:
4         print(x)
```

```
1
2
3
4
5
6
7
8
9
```

- Use **end** to get a beautiful matrix
  - with space or a special character

```
1 list_of_lists = [ [1,2,3], [4,5,6], [7,8,9]]
2 for list in list_of_lists:
3     for x in list:
4         print(x, end=" ")
5     print()
```

```
1 2 3
4 5 6
7 8 9
```

```
\0 Null character
\t Horizontal tab
\n Newline character
\' Single quote
\" Double quote
\\ Backslash
```

Python basics: tuple, set & dictionary

- Tuple ( )
  - ordered & unchangeable collection
- Set { }
  - unordered & changeable
  - duplicates are not allowed
- Dictionary { } key/value-pairs
  - ordered & changeable collection
  - duplicates are not allowed

```python
tuple = ("spring","summer","autumn","winter")
print(tuple)

set = {"apple", "banana", "cherry"}
set.add("mango")
set.add("apple")
print(set)

dictionary = {
    "a":"apple",
    "b" : "banana",
    "c" : "cherry",
    "c" : "citron"
}
print(dictionary)
dictionary["y"] = "cherry"
print(dictionary)
```

```
('spring', 'summer', 'autumn', 'winter')
{'mango', 'cherry', 'banana', 'apple'}
{'a': 'apple', 'b': 'banana', 'c': 'citron'}
{'a': 'apple', 'b': 'banana', 'c': 'citron', 'y':
'cherry'}
```

01 Python basics

- ~~Why Python?~~
- ~~The basics!~~
- ~~Flow control~~
- ~~Functions~~
- ~~Collections: lists, tuple, set & dictionary~~
- Arrays
- Pythonic code

Python basics: using libaries!

- Python does not have built-in support for arrays!
- If we need an array (fixed collection of data)

  👉 **import a library to work with arrays!**
- Libraries are the **power of Python**!
- Where to find all libraries? https://pypi.org/
  - Search on any topic!
  - Read the documentation, does it suit your needs?
- How to use a library?
  - Install library (pip-command!) **pip install libray**
  - Import library in script **import library**

from hell import depression as magic

Python basics: numpy library

- **NumPy** is a library used for working with **arrays**!
- Take a look at the documentation
  - https://pypi.org/project/numpy/
  - https://numpy.org/
- Install **NumPy**

```
PS C:\> pip install numpy
```

- Different options when you **import** a library
  - import whole package
  - import whole package and rename
  - import a single functionality from the package

```
1 import numpy
2 import numpy as np
3 from numpy import arange
```

Python basics: numpy library

- Create an array by calling one of the many functions available in the library
- Once an **array** is created, its **fixed size**!
- **Array elements** all have the **same type**!

```python
import numpy as np #import library

list = [1,2,3,4,5]
print(list)
array = np.array([1,2,3,4,5])
print(array)

print(list*2)   #look at the difference!
print(array*2)  #same operator, different result

print(np.arange(10))
print(np.zeros((10,)))
print(np.ones((2,3)))
print(np.full((5,), 4))
print(np.eye(4))
print(np.random.random((1,4)))
print(np.random.random_integers(-10, 10, (4,4)))
```

```
[1, 2, 3, 4, 5]
[1 2 3 4 5]
[1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
[ 2  4  6  8 10]


[0 1 2 3 4 5 6 7 8 9]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
[[1. 1. 1.]
 [1. 1. 1.]]
[7 7 7 7 7]
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
[[0.34400512 0.04234007 0.99449629 0.08878798]]
[[ -9   1  -9  10]
 [  9 -10   9   3]
 [ 10  -9  -2   5]
 [  6   6   5  10]]
```

Python basics: numpy library

- Get type of elements by use of **dtype**
- **Convert** elements of array to another type **astype**
- **Reshape** an array

```
array = np.array([1, 2, 3, 4, 5, 6,7,8])
print(array)
print(type(array))                #mNumpy array
print(array.dtype)
new_array = array.astype('f')
print(new_array)
print(new_array.dtype)

print("SHAPE: " + str(array.shape) + " DIM: " + str(array.ndim))

array_2D = np.reshape(array, (4,2))
print(array_2D)
print("SHAPE: " + str(array_2D.shape) + " DIM: " + str(array_2D.ndim))
array_3D = np.reshape(array, (2,2,2))
```

```
[1 2 3 4 5 6 7 8]
<class 'numpy.ndarray'>
int32

[1. 2. 3. 4. 5. 6. 7. 8.]
float32

SHAPE: (8,) DIM: 1
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
SHAPE: (4, 2) DIM: 2
```

Python basics: numpy library

- **Slicing** = taking elements from a startindex to an endindex [start: end]
  - define stepsize [start: end: step]
  - empty start index = 0
  - empty end index = array end (step as 1)
  - we can use negative indexes!

```
a = np.arange(8)**3    # exponent 3 --> 0^3 1^3 2^3 3^3 ...
print(a)
print(a[2:5])
print(a[:4])
print(a[4:])
a[::3] = -1
print(a)
print(a[::-1])  #reverse
print(a[1::2])  # odd indexes
```

```
[  0   1   8  27  64 125 216 343]
[ 8 27 64]
[ 0  1  8 27]
[ 64 125 216 343]
[ -1   1   8  -1  64 125  -1 343]
[343  -1 125  64  -1   8   1  -1]
[  1  -1 125 343]
```

Python basics: numpy library

- **A lot of useful methods!**

```
a = np.arange(0,10)
b = np.arange(9,-1,-1)
print(a)
print(b)
c = a-b
print(c)
print(c<0)
print(np.where(c<0))
c %= 3
print(c)
print(np.add(a, c))

#if condition == true, a+1, else a
print(np.where(a%2==1,a+1,a))

b = np.reshape(b,(2,5))
print(b)
print(b.sum(axis=0))          # sum of each column
print(b.min(axis=1))          # min of each row
print(b.cumsum(axis=1))       # cumulative sum along each row
print(np.sqrt(b))
```

```
[0 1 2 3 4 5 6 7 8 9]
[9 8 7 6 5 4 3 2 1 0]
[-9 -7 -5 -3 -1  1  3  5  7  9]
[ True  True  True  True  True False False False False False]
(array([0, 1, 2, 3, 4], dtype=int32),)
[0 2 1 0 2 1 0 2 1 0]
[0 3 3 3 6 6 6 9 9 9]
[ 0  2  2  4  4  6  6  8  8 10]
[[9 8 7 6 5]
 [4 3 2 1 0]]
[13 11  9  7  5]
[5 0]
[[ 9 17 24 30 35]
 [ 4  7  9 10 10]]
[[3.         2.82842712 2.64575131 2.44948974 2.23606798]
 [2.         1.73205081 1.41421356 1.         0.        ]]
```

01 Python basics

- ~~Why Python?~~
- ~~The basics!~~
- ~~Flow control~~
- ~~Functions~~
- ~~Collections: lists, tuple, set & dictionary~~
- ~~Arrays~~
- Pythonic code

Pythonic code

“ In the Python community, Pythonic is the word describing code that doesn't just get the syntax right but uses the language in the way it is intended to be used. It improves the overall code quality from maintainability, readability and efficiency perspective.

Zen of Python

- *Collection of 19 guiding principles*
- **Import this** in your Python code

```
import this
```

```
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

Example list

Not Pythonic

```python
data = ["apple","banana","mango","kiwi"]

# Simple looping of list
for i in range(0, len(data)):
    print(data[i])

# Looping with index
for i in range(len(data)):
    print(i,data[i])

# Looping reverse list
for i in range(len(data)-1, -1, -1):
    print(data[i])
```

Pythonic

```python
data = ["apple","banana","mango","kiwi"]

# Simple looping of list
for val in data:
    print(val)

# Looping with index
for i, val in enumerate(data):
    print(i, val)

# Looping reverse list
for val in reversed(data):
    print(val)
```

Source: *https://nunenuh.medium.com/how-to-write-pythonic-code-e19b1562fd5*

Example dictionary

```
data = ["apple","banana","mango","kiwi"]
colors = ["green", "yellow","orange", "green"]
```

Not Pythonic

```
# Create a dictionary of pairs
dc = {}
for key, val in zip(data, colors):
    dc[key] = val

# Loop dictionary
for key in dc:
    print(key, dc[key])
```

Pythonic

```
# Create a dictionary of pairs
dc = dict(zip(data, colors))


# Loop dictionary
for key, val in dc.items():
    print(key, val)
```

Source: *https://nunenuh.medium.com/how-to-write-pythonic-code-e19b1562fd5*

Example longest palindrome

Not Pythonic

```
longest = ""
for word in wordlist:
    is_palindrome = True
    for index in range(len(word)):
        if word[index] != word[-(index + 1)]:
            is_palindrome = False
    if is_palindrome and len(word) > len(longest):
        longest = word
print(longest)
```

Pythonic

```
longest = ""
for word in wordlist:
    if word == word[::-1] and len(word) > len(longest):
        longest = word
print(longest)
```

On steroids ;)

```
max([word for word in wordlist if word == word[::-1]], key=len)
```

But is this readable?

Practice makes perfect!

- *Check out the extra (not graded) exercises in the notebooks!* *See Github*
- Do your exercises, spend the hours!

Say what? How many hours?

6 SP = 6 * 28 hours = 168 hours

Lessons = 12 * 3 hours = 36 hours

Take-home exam = 16 hours

Exercise = 168-36-10 = 116 hours